RT Crackers Mail

Feature & Code Guide

A Gmail / Outlook-style business webmail for the 10 RT Crackers mailboxes

FastAPI - Supabase Postgres - Resend - vanilla JavaScript

Version 1.2 | October 2026 | Database schema: unchanged

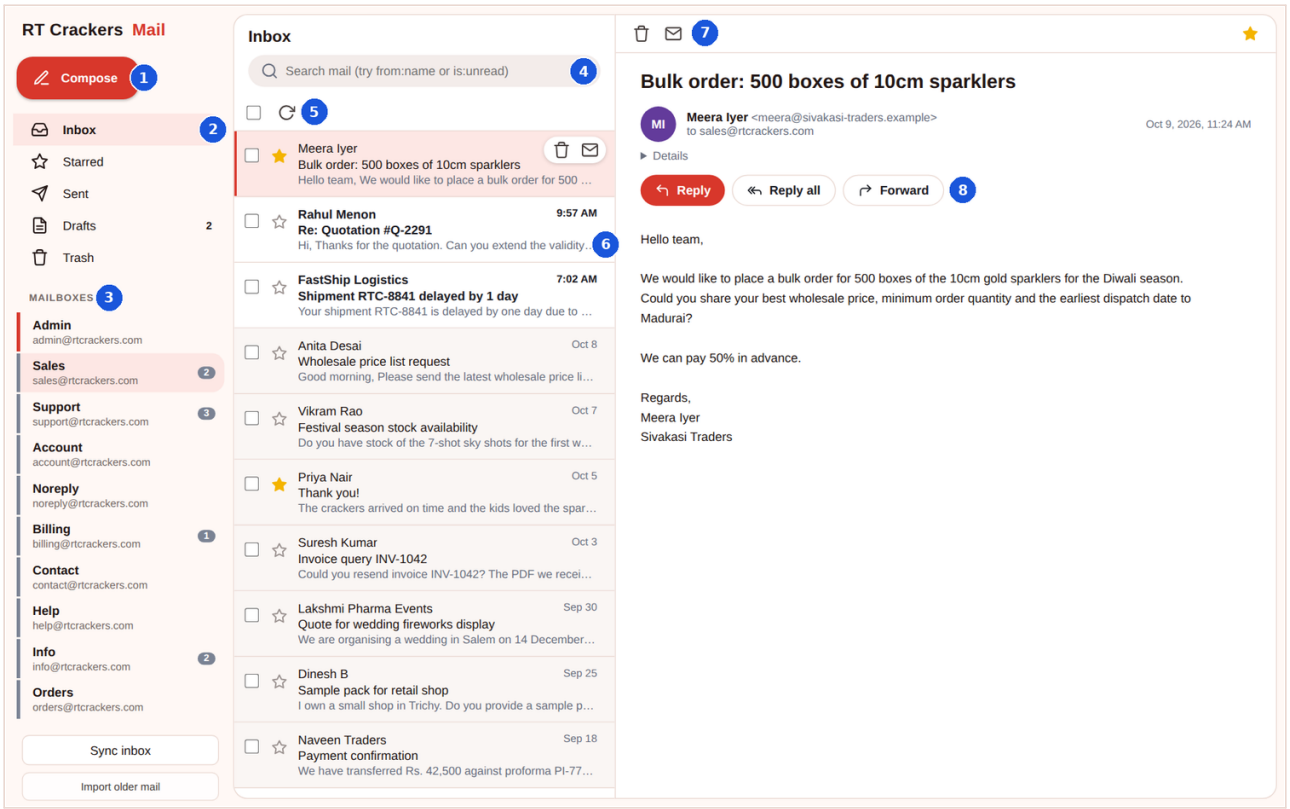


Figure 1. The finished interface: folders, mailboxes, message list, reader. Numbers are explained in section 4.1.

Contents

1. Overview . . . 3
What changed in this version . . . 3
At a glance . . . 3

2. Complete feature list . . . 4
Reading and navigation . . . 4
Organizing mail . . . 4
Writing mail . . . 4
Finding mail . . . 5
Delivery, sync and reliability . . . 5
Privacy and security . . . 5
Speed and efficiency . . . 5
Productivity . . . 6
Gmail / Outlook parity - an honest comparison . . . 6

3. Architecture . . . 6
Modules . . . 7
Life of a message . . . 8

4. Feature guide with code . . . 9
4.1 Three-pane layout and folders . . . 9
4.2 Stars . . . 10
4.3 Checkboxes, bulk actions and hover actions . . . 12
4.4 Trash, Undo, Restore and Delete forever . . . 14
4.5 Drafts with autosave . . . 15
4.6 Reply, Reply all and Forward . . . 18
4.7 Search operators and Load more . . . 19
4.8 Delivery status . . . 21
4.9 Privacy: remote images blocked . . . 22
4.10 Keyboard shortcuts . . . 23
4.11 Phone layout . . . 25
4.12 Mailboxes, login and access control . . . 26
4.13 Speed: why the site loads fast now . . . 27

5. API reference . . . 28

6. Database usage - no schema changes . . . 29

7. Quality and testing . . . 29
Real-browser verification . . . 29

8. Setup and deployment . . . 30
Deploy . . . 30

9. Limitations and suggested next steps . . . 30

Appendix: project files . . . 30

1. Overview

RT Crackers Mail is a private webmail portal for the business mailboxes **admin, sales, support, account, noreply, billing, contact, help, info** and **orders** at the company domain. Mail is sent and received through **Resend**, stored in your **Supabase Postgres** database, and protected by **Supabase Auth**. The server is a small **FastAPI** application; the interface is plain HTML, CSS and JavaScript with no build step, so it deploys to Vercel as-is.

This release turns the original inbox viewer into a complete mail client that behaves like Gmail and Outlook: stars, trash with undo, bulk actions, drafts with autosave, reply / reply all / forward, search operators, paging, delivery status, keyboard shortcuts and a phone layout. This guide lists every feature, explains how each one works, and quotes the real code behind it.

What changed in this version

Stage	What was added
Code review	Fixed sync, read-state, token-refresh, race-condition and header-injection bugs; added input limits, security headers, JSON error handling, safer config.
Privacy	Remote images are blocked by default; HTML mail renders in a sandboxed iframe with its own Content-Security-Policy.
Drafts	Autosaving drafts, Drafts folder, resume editing, send-from-draft, delete draft.
Reply tools	Reply all and Forward; HTML-only mail is quoted as clean plain text.
Speed	One-request start-up (bootstrap), instant skeleton screen, cached mailboxes, one-query sync check, lightweight new-mail check (pulse), cacheable static files. Measured: first list shown 4.7x faster; sync of 100 stored emails 102 connections down to 1.
Gmail / Outlook features	Starred, Trash (restore, delete forever, empty), Undo, mark read/unread, checkboxes with bulk actions, hover quick actions, search operators, Load more paging, delivery status chips, keyboard shortcuts, mobile single-pane layout.

At a glance

Measure	Value
Backend tests (real PostgreSQL)	**141** passing (unit, service layer, HTTP API, webhooks)
Frontend behaviour checks (jsdom)	**54** passing (drafts, bulk actions, trash, paging, shortcuts, mobile flow)
Real-browser verification	Chromium run against the real backend + database; found and fixed two UI bugs the simulated tests could not see
Application code	1488 lines Python, 1307 lines HTML/CSS/JS
Database schema	**Unchanged.** db.py, supabase/*.sql and tests/schema.sql are byte-identical to the original upload; all new features use columns that already existed
Third-party front-end libraries	None

2. Complete feature list

Every feature below is implemented and covered by automated checks. The last column names where to find it in the code (all paths are relative to the project root).

Reading and navigation

Feature	What it does	Where in the code
Three-pane layout	Folders | message list | reader, like Gmail and Outlook. Rounded panels, pill Compose button.	public/index.html, style.css
Folders with counts	Inbox (unread count), Starred, Sent, Drafts (count), Trash; active folder highlighted.	app.js: setView, renderNav
Mailbox switcher	All 10 business mailboxes with unread badges; the folder you are in is kept when you switch.	app.js: renderBoxes, selectBox
Smart dates	Time for today, "Oct 8" for this year, short date for older mail.	app.js: fmtListDate
Unread styling	Unread rows are bold on white, read rows are tinted; opening a message marks it read instantly.	app.js: rowNode, markRowOpen
Reader header	Avatar with initials, sender, "to ..." line, expandable Details, status chip on sent mail.	app.js: renderReader
Phone layout	One pane at a time with a Back button and floating Compose button.	style.css: @media, app.js: closeReader

Organizing mail

Feature	What it does	Where in the code
Star / unstar	Click the star on a row or in the reader (or press S). Starred folder lists starred inbox and sent mail.	service.update_messages, app.js: toggleStar
Checkboxes and bulk actions	Select rows or select all (with indeterminate state); trash, mark read/unread, star in one click.	app.js: renderToolbar, bulk
Hover quick actions	Trash and mark read/unread appear on a row when you hover it.	app.js: rowNode
Trash and Undo	Moving to Trash is reversible; a toast offers Undo. Trash also has Restore, Delete forever and Empty Trash.	service._ACTION_SQL, app.js: bulk
Mark read / unread	From the toolbar, hover actions, reader or keyboard (Shift+I / Shift+U).	service.update_messages
Instant feedback	Lists update immediately; stars revert automatically if the server rejects the change.	app.js: applyLocal, toggleStar

Writing mail

Feature	What it does	Where in the code
Compose	From picker, To / Cc / Bcc, subject, body; Ctrl+Enter sends.	app.js: renderCompose
Autosaving drafts	Saved about 1.5 s after you stop typing, when you leave the screen, or when the tab is hidden.	service.save_draft, app.js: renderCompose
Drafts folder	Reopen, edit, send or delete a draft; sending removes it; a failed send keeps it.	service.send_mail, delete_draft
Reply, Reply all, Forward	Quoted original, correct To/Cc (own address and duplicates removed), "Fwd:" header block.	app.js: reply, forward
Clean HTML quoting	HTML-only mail is converted to plain text with an inert parser before quoting or forwarding.	app.js: plainText

Finding mail

Feature	What it does	Where in the code
Search operators	from:, to:, subject:, is:unread, is:read, is:starred, combinable with free text; wildcards in the query stay literal.	service.parse_search, list_messages
Load more paging	50 messages per page, up to 200 per request; duplicates are never shown.	routes.list_messages, app.js: loadList
Race-safe loading	A slow response can never overwrite a newer list.	app.js: loadList (request counter)

Delivery, sync and reliability

Feature	What it does	Where in the code
Delivery status	Sent mail shows Delivered, Bounced, Failed or Spam report (from Resend webhooks); a late event never hides a bounce.	service.apply_delivery_event, routes.resend_webhook
Receive mail	Instant via webhook, plus a 60 s auto-sync and Import older mail; only one sync runs at a time.	service.sync_inbox, ingest_received
Rate-limited sending	Resend client throttles to the API limit and retries on 429 with back-off.	mailcore/resend_client.py
Sent-but-not-saved guard	If mail is delivered but cannot be stored, you get a clear error and the draft is kept.	service.send_mail

Privacy and security

Feature	What it does	Where in the code
Remote images blocked	Tracking pixels cannot fire; a banner offers Load images per message.	app.js: renderReader
Sandboxed mail view	HTML mail runs in a sandboxed iframe with scripts disabled and its own CSP.	app.js: renderReader
Security headers	CSP, X-Frame-Options, nosniff, referrer and permissions policy on every response; private API responses are no-store.	main.py, routes.py
Login and access control	Supabase sign-in, single-flight token refresh, per-mailbox permissions enforced in SQL.	mailcore/auth.py, service.py
Input limits	Field lengths, recipient counts, page size and bulk size are validated.	routes.py (Pydantic), service.py

Speed and efficiency

Feature	What it does	Where in the code
One-request start-up	Mailboxes, unread/draft counts and the first page of mail arrive in a single request; a skeleton shows instantly.	service.bootstrap, routes.bootstrap, app.js: start
New-mail check	A tiny request every 20 s shows mail the webhook already stored; the list reloads only when something is new.	service.pulse, app.js: pollPulse
One-query sync check	A sync asks once per page which emails are already stored (it used one connection per email).	service._load_known, sync_inbox
In-memory caches	Mailboxes (2 min), the mailbox-creation check (10 min) and verified sessions (5 min) are kept in memory.	service.list_mailboxes, auth.py
Local count updates	Opening or trashing mail adjusts badges on screen instead of re-fetching counts; bursts are merged into one refresh.	app.js: scheduleRefresh, openMessage
Cacheable static files	Browser and Vercel's CDN keep the page, script and stylesheet, so loading them does not wake the Python function.	main.py: SecurityHeadersMiddleware

Productivity

Feature	What it does	Where in the code
Keyboard shortcuts	C, /, J/K, Enter/O, X, S, #, Shift+I/U, R/A/F, U/Esc, G then I/S/T/D/B, ? for help.	app.js: onKey, SHORTCUTS
Shortcut help overlay	Press ? anywhere in the list or reader.	app.js: toggleHelp

Gmail / Outlook parity - an honest comparison

Capability	This app	Notes
Folders, stars, trash, undo, bulk actions	Yes	Behaves like Gmail; trash is permanent after Delete forever.
Search operators	Partial	from:, to:, subject:, is:unread/read/starred. No date, size or attachment operators.
Drafts with autosave	Yes	Recipients are stored only once an address is complete.
Reply, Reply all, Forward	Yes	Forward does not carry attachments (the app has no attachment support yet).
Conversation threading	Not yet	Messages are listed individually; reply headers are kept so threads stay intact in other clients.
Attachments (send or view)	Not yet	Listed in the roadmap.
Custom labels, archive, snooze	Not yet	Not part of this release.
Rich-text composer, signatures	Not yet	Compose is plain text (sent as text + HTML).
Dark mode, offline mode	Not yet	-

3. Architecture

The browser talks only to the FastAPI server over JSON. The server owns every connection to the database, to Supabase Auth and to Resend, so no secret or database URL ever reaches the page. Resend pushes events (new mail, delivery results) back to the server through a signed webhook.

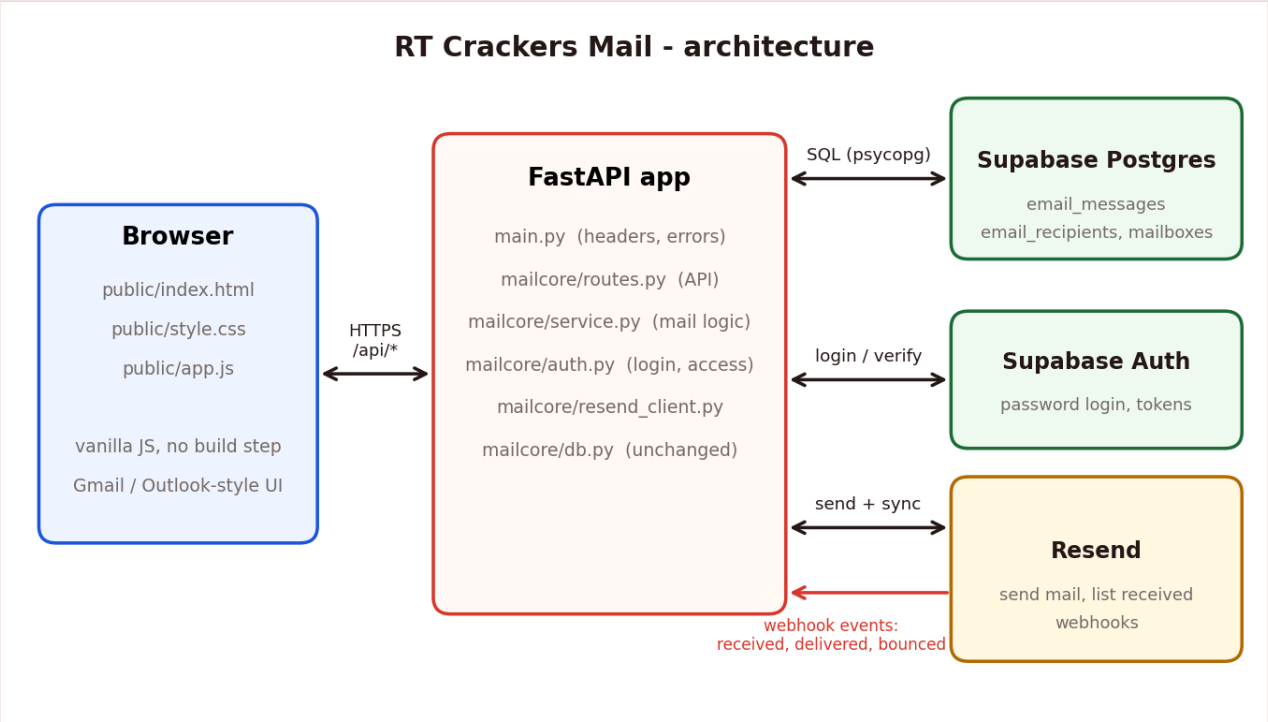


Figure 2. How the pieces fit together.

Modules

File	Lines	Responsibility
main.py	101	Creates the app; adds security headers; turns database and configuration errors into clean JSON; serves the front end.
mailcore/routes.py	345	The HTTP API and the Resend webhook; validates input with Pydantic.
mailcore/service.py	718	All mail logic: mailboxes, sync, search, send, drafts, star/trash actions, delivery status.
mailcore/auth.py	144	Supabase login/refresh, permission lookup, short-lived token cache.
mailcore/resend_client.py	92	Rate-limited Resend REST client with retry.
mailcore/db.py	46	Postgres connection helper. **Unchanged from the original.**
mailcore/config.py	42	Environment variables with safe parsing.
public/app.js	1025	The whole interface: list, reader, compose, drafts, bulk actions, shortcuts.
public/index.html, style.css	68 + 214	Markup and styling, including the phone layout.

Life of a message

Every state below is stored in columns that already exist on `email_messages`, which is why no migration is needed.

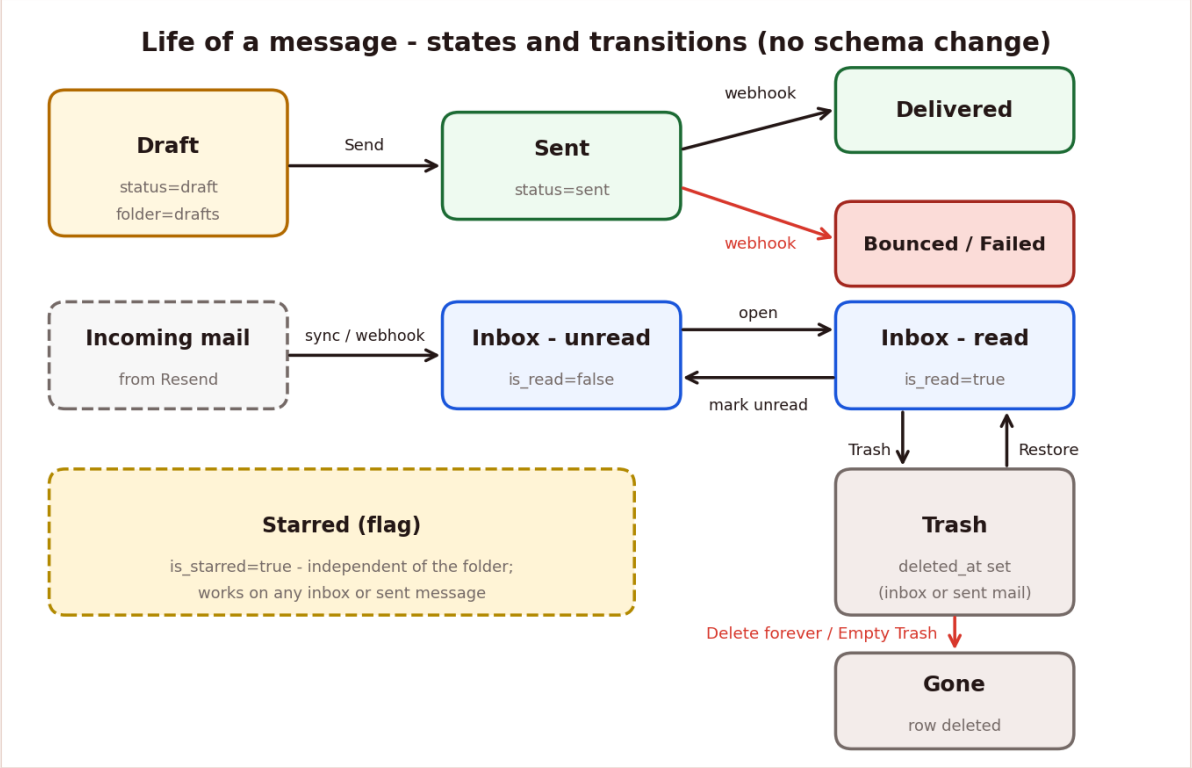


Figure 3. States and transitions of a message.

4. Feature guide with code

Each section shows the feature, how to use it, how it works, and the actual code (copied from the repository with its line numbers).

4.1 Three-pane layout and folders

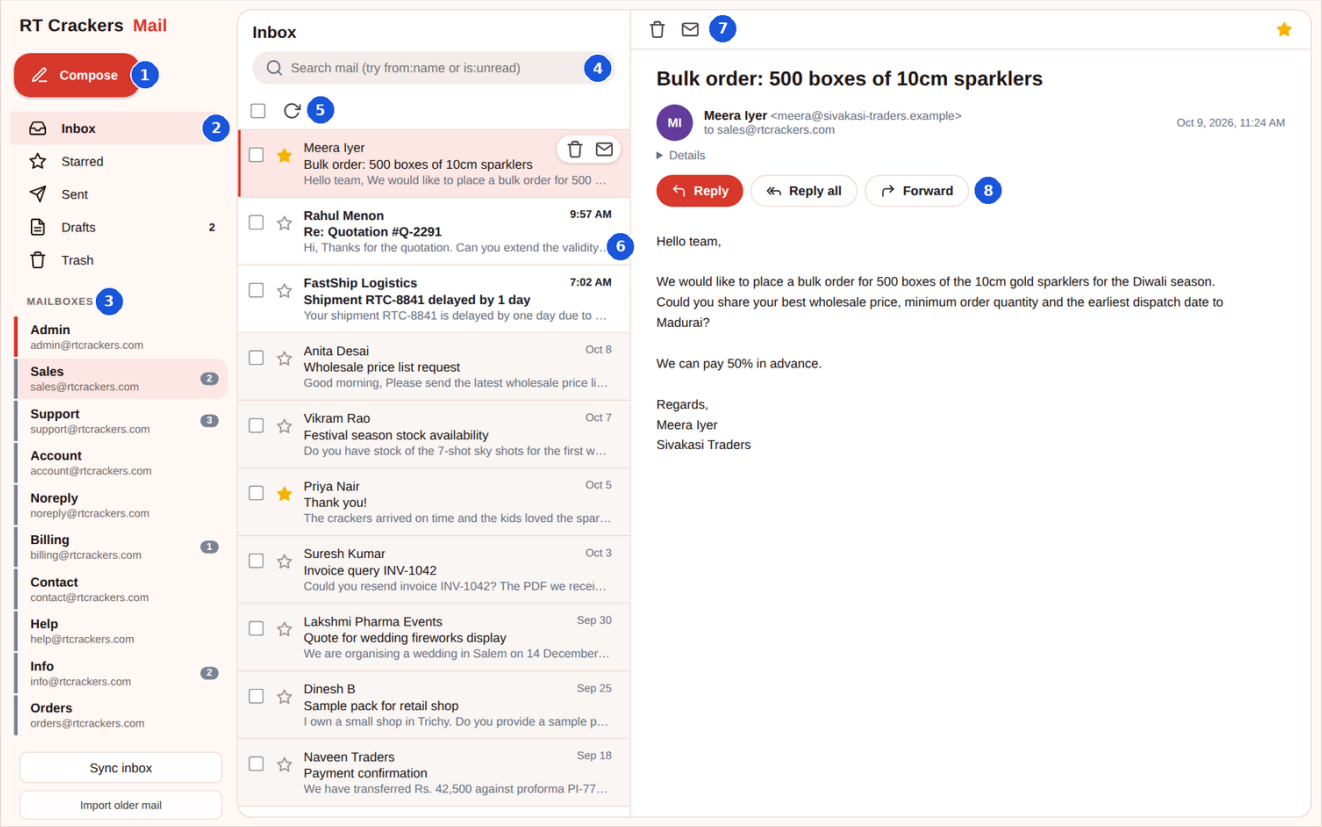


Figure 4. The main screen. Numbers are explained below.

1	**Compose** - opens the editor (shortcut C).
2	**Folders** - Inbox, Starred, Sent, Drafts, Trash with live counts.
3	**Mailboxes** - switch between the 10 business addresses; badges show unread mail.
4	**Search** - supports operators such as from: and is:unread.
5	**List toolbar** - select all and refresh; turns into bulk actions when rows are selected.
6	**Message row** - checkbox, star, sender, subject, preview, smart date.
7	**Reader toolbar** - trash, mark unread, star.
8	**Reply, Reply all, Forward.**

How it works

One function, `setView`, switches folders: it saves any unsaved draft first, resets the list state, highlights the folder, and loads the first page. The list is rendered from a single in-memory array, so selection, stars and counts always agree with what is on screen.

public/app.js (lines 734-753) - switching folders

```
function setView(view) {
  const wasComposing = S.view === "compose";
  if (view === "compose") {
    enterCompose();
    return renderCompose({ from: S.box });
  }
  const saving = leaveCompose();   // saves unsaved edits as a draft before leaving
  S.view = view;
  S.openId = null; S.openMsg = null;
  S.items = []; S.sel = new Set(); S.cur = -1; S.hasMore = false; S.loading = true;
  renderNav();
  $("#app").classList.remove("composing", "reading");
  $("#listTitle").textContent = TITLES[view] || "Inbox";
  readerPlaceholder();
  renderRows();
  saving.then(() => {
    if (wasComposing) scheduleRefresh();   // draft counts may have changed
    if (S.view === view) loadList();     // ignore if the user already moved on
  });
}
```

public/app.js (lines 151-157) - Gmail-style dates

```
function fmtListDate(iso) {
  const d = new Date(iso), n = new Date();
  if (isNaN(d)) return "";
  if (d.toDateString() === n.toDateString()) return d.toLocaleTimeString([], { hour: "numeric",
      minute: "2-digit" });
  if (d.getFullYear() === n.getFullYear()) return d.toLocaleDateString([], { month: "short", day:
      "numeric" });
  return d.toLocaleDateString([], { year: "2-digit", month: "numeric", day: "numeric" });
}
```

4.2 Stars

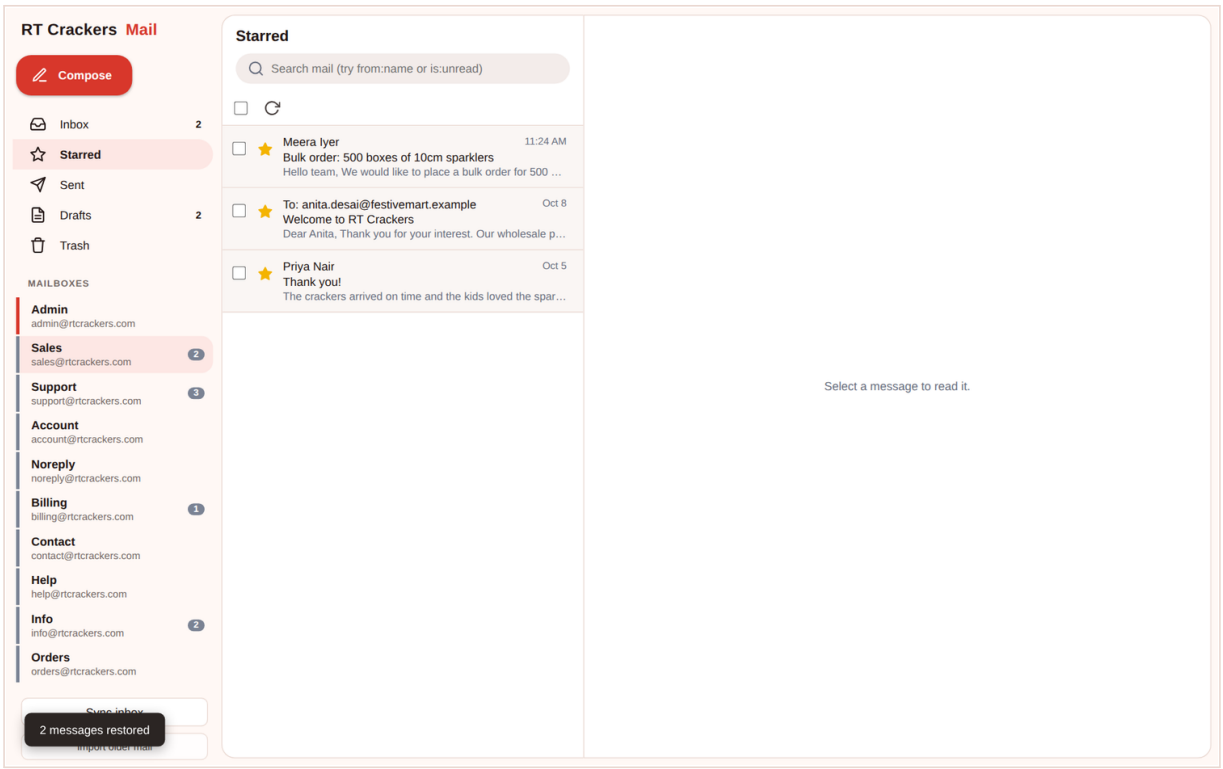


Figure 5. The Starred folder lists starred mail from both Inbox and Sent.

Use it	Click the star on a row or in the reader, or press S. Open Starred to see them all.
Like	Gmail Starred, Outlook Flag.
Stored in	email_messages.is_starred (existing column)
Tests	service star/unstar, starred view incl. sent mail, API bulk flow, browser star + Starred folder

How it works

The browser flips the star immediately (optimistic update) and sends the change to the server; if the server refuses, the star flips back and an error toast appears. On the server a single SQL statement updates only rows that actually change and only in mailboxes the user may access, so the reply tells the browser how many changed.

public/app.js (lines 432-442) - optimistic update with automatic revert

```
async function toggleStar(id) {
  const m = S.items.find((x) => x.id === id) || (S.openMsg && S.openMsg.id === id ? S.openMsg :
      null);
  const on = !(m && m.is_starred);
  setStarLocal(id, on);   // optimistic
  try {
    await api("/messages/bulk", { method: "POST", body: JSON.stringify({ ids: [id], action: on ?
        "star" : "unstar" }) });
  } catch (e) {
    setStarLocal(id, !on);
    toast(e.message, true);
  }
}
```

mailcore/service.py (lines 623-624) - the SQL behind star / unstar

```
    "star": "update email_messages set is_starred = true, updated_at = now() where folder in
        ('inbox','sent') and not is_starred",
    "unstar": "update email_messages set is_starred = false, updated_at = now() where folder in
        ('inbox','sent') and is_starred",
```

mailcore/service.py (lines 631-645) - permission-scoped bulk update (used by every action in this chapter)

```
def update_messages(user, ids: list[int], action: str) -> int:
    """Apply `action` to the given messages the user can access. Returns how many actually
        changed.

    Drafts are never touched here (use delete_draft). delete_forever only removes messages already
         in Trash.
    """
    if action not in _ACTION_SQL:
        raise ValueError("Unknown action")
    ids = list({int(i) for i in ids})
    if not ids:
        return 0
    if len(ids) > MAX_PAGE:
        raise ValueError(f"Too many messages selected (maximum {MAX_PAGE})")
    sql = _ACTION_SQL[action] + " and email_message_id = any(%s) and mailbox_id = any(%s)
        returning email_message_id"
    with conn() as c:
        return len(c.execute(sql, (ids, _accessible_ids(user))).fetchall())
```

4.3 Checkboxes, bulk actions and hover actions

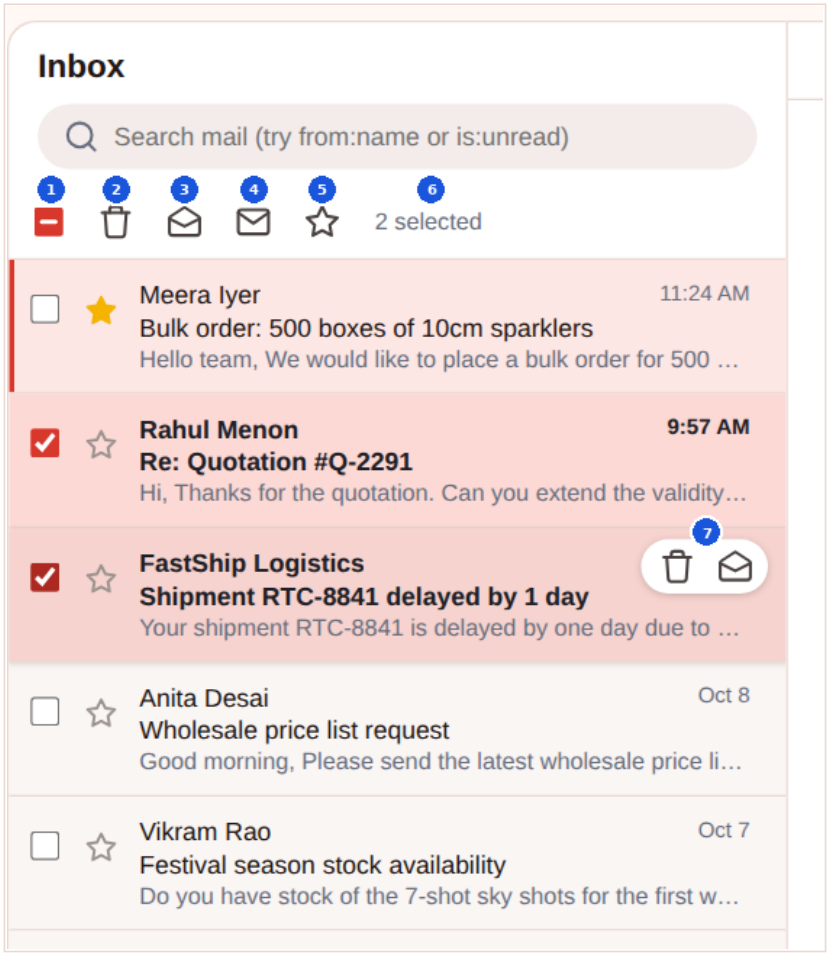


Figure 6. Two rows selected: the toolbar switches to bulk actions. The hovered row shows quick actions.

1	**Select all** - shows a partial state when only some rows are selected.
2	**Trash**
3	**Mark as read**
4	**Mark as unread**
5	**Star**
6	**Selection count**
7	**Hover quick actions** - trash and mark read/unread without opening the message.

Use it	Tick rows (or press X), then choose an action. Hover a row for one-click trash and read/unread.
Like	Gmail and Outlook list toolbars.
Limits	Up to 200 messages per action (validated on server).
Tests	browser: select two rows, bulk trash, undo; service: bulk permissions and limits

How it works

`bulk()` is the single entry point for every action on one or many messages. It asks for confirmation on destructive actions, calls the API, updates the on-screen list at once, refreshes the counters, and shows a toast. For Trash the toast carries an **Undo** button that simply sends the opposite action.

public/app.js (lines 393-418) - one function for every message action

```
async function bulk(action, ids) {
  if (!ids.length) return 0;
  const n = ids.length, plural = n === 1 ? "" : "s";
  if (action === "delete_forever" && !confirm(`Permanently delete ${n} message${plural}? This
      cannot be undone.`)) return 0;
  if (action === "delete_draft" && !confirm(`Delete ${n} draft${plural}?`)) return 0;
  let changed = 0;
  try {
    if (action === "delete_draft") {
      for (const id of ids) { try { await api(`/drafts/${id}`, { method: "DELETE" }); changed++; }
           catch {} }
    } else {
      changed = (await api("/messages/bulk", { method: "POST", body: JSON.stringify({ ids, action
          }) })).updated;
    }
  } catch (e) { toast(e.message, true); return 0; }

  S.sel = new Set();
  if (S.openId && ids.includes(S.openId) && ["trash", "restore", "delete_forever",
      "unread"].includes(action)) closeReader();
  applyLocal(action, ids);
  scheduleRefresh();

  if (action === "trash") {
    toast(`${n} message${plural} moved to Trash`, false, { label: "Undo", run: async () => { await
        bulk("restore", ids); loadList(); } });
  } else if (action === "restore") toast(`${n} message${plural} restored`);
  else if (action === "delete_forever") toast(`${n} message${plural} deleted forever`);
  else if (action === "delete_draft") toast(`${n} draft${plural} deleted`);
  return changed;
}
```

mailcore/routes.py (lines 231-235) - the API endpoint (input is validated by the BulkIn model)

```
@router.post("/messages/bulk")
def bulk_update(body: BulkIn, response: Response, user: auth.User = Depends(auth.current_user)):
    """Star, trash, restore, mark read/unread or permanently delete several messages at once."""
    response.headers["Cache-Control"] = "no-store"
    return {"updated": service.update_messages(user, body.ids, body.action)}
```

4.4 Trash, Undo, Restore and Delete forever

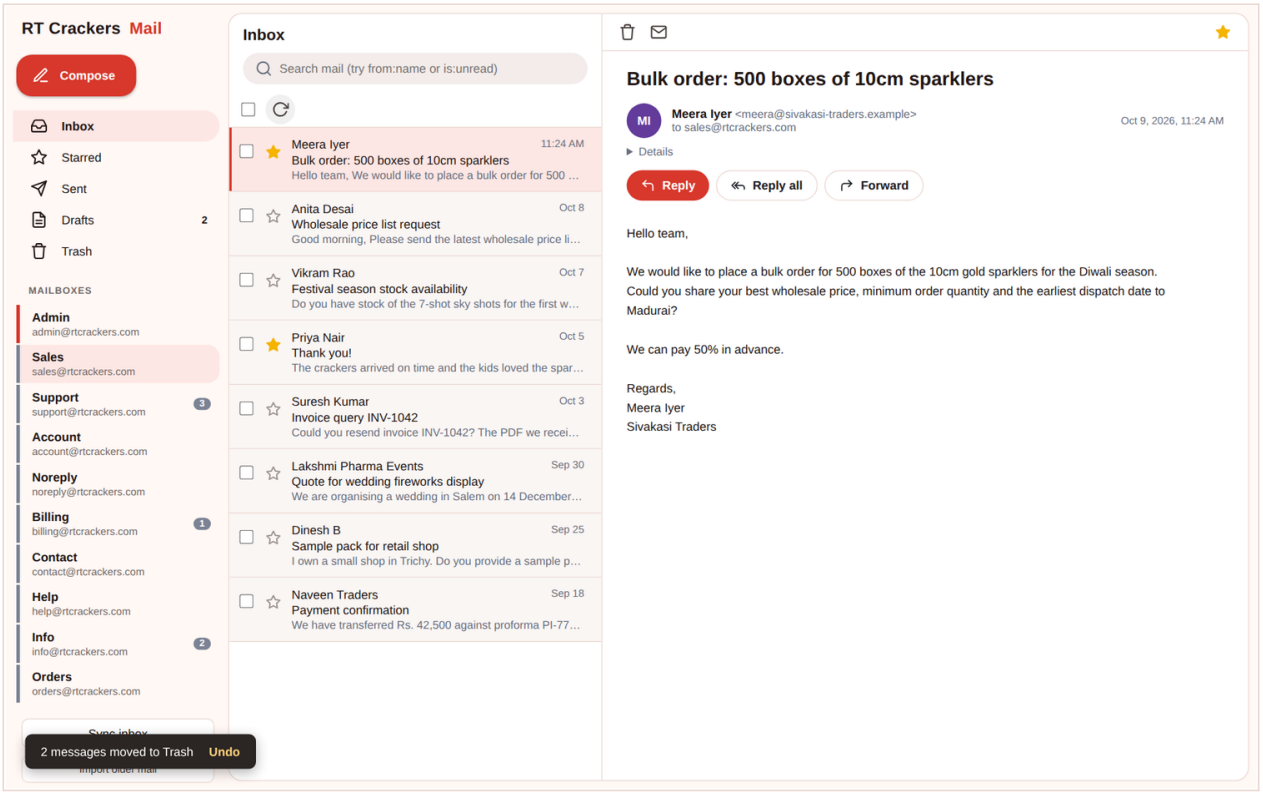


Figure 7. After moving two messages to Trash, a toast offers Undo.

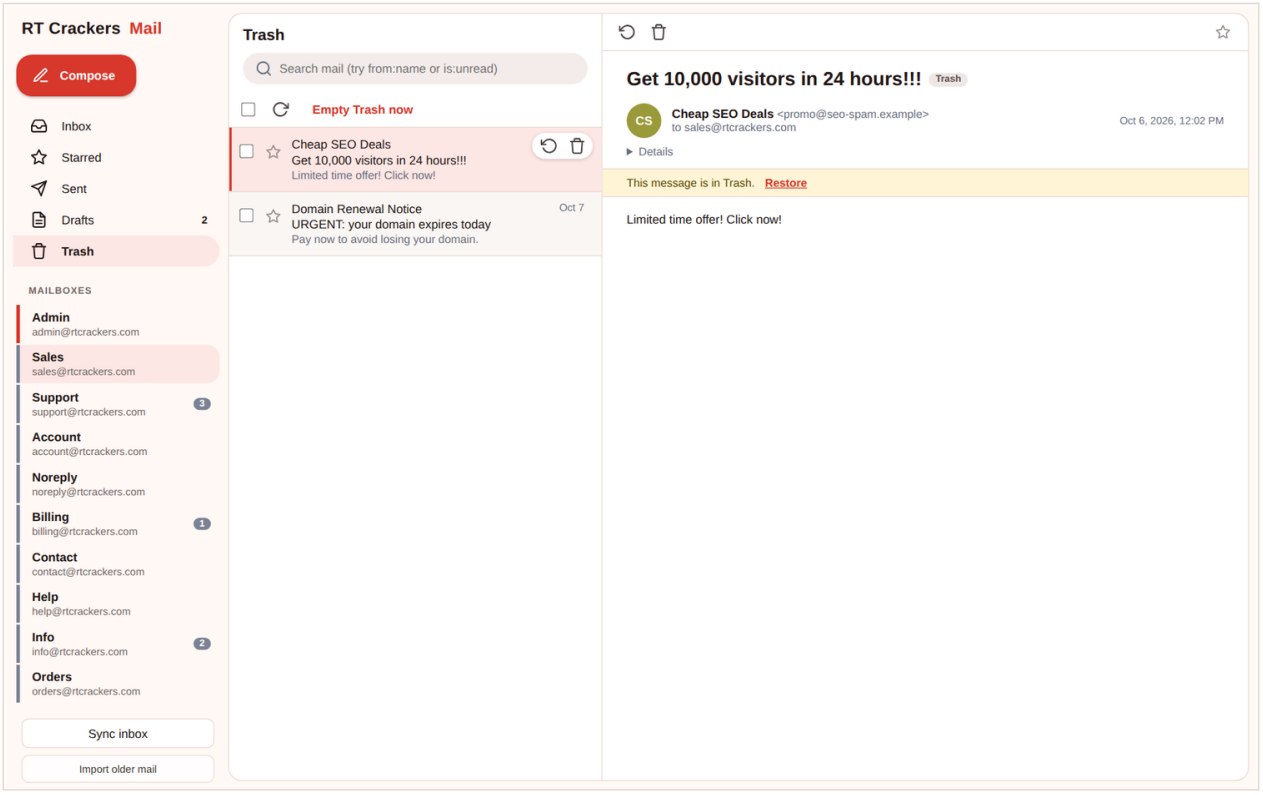


Figure 8. A message opened from Trash: a banner offers Restore and the reply buttons are hidden.

Use it	Trash (button, hover icon or #). Undo within a few seconds, or open Trash to Restore, Delete forever, or Empty Trash now.
Like	Gmail Trash, Outlook Deleted Items.

Stored in	email_messages.deleted_at - a soft delete; Restore simply clears it.
Safety	Delete forever only removes messages that are already in Trash; unread counts ignore trashed mail; viewing a trashed message does not mark it read.

mailcore/service.py (lines 625-627) - trash, restore and delete-forever statements

```
    "trash": "update email_messages set deleted_at = now(), updated_at = now() where folder in
        ('inbox','sent') and deleted_at is null",
    "restore": "update email_messages set deleted_at = null, updated_at = now() where deleted_at
        is not null",
    "delete_forever": "delete from email_messages where deleted_at is not null",
```

mailcore/service.py (lines 648-655) - Empty Trash is limited to one mailbox

```
def empty_trash(user, local: str) -> int:
    box = _box_by_local(local, user)
    with conn() as c:
        rows = c.execute(
            "delete from email_messages where mailbox_id = %s and deleted_at is not null returning
                email_message_id",
            (box["mailbox_id"],),
        ).fetchall()
    return len(rows)
```

4.5 Drafts with autosave

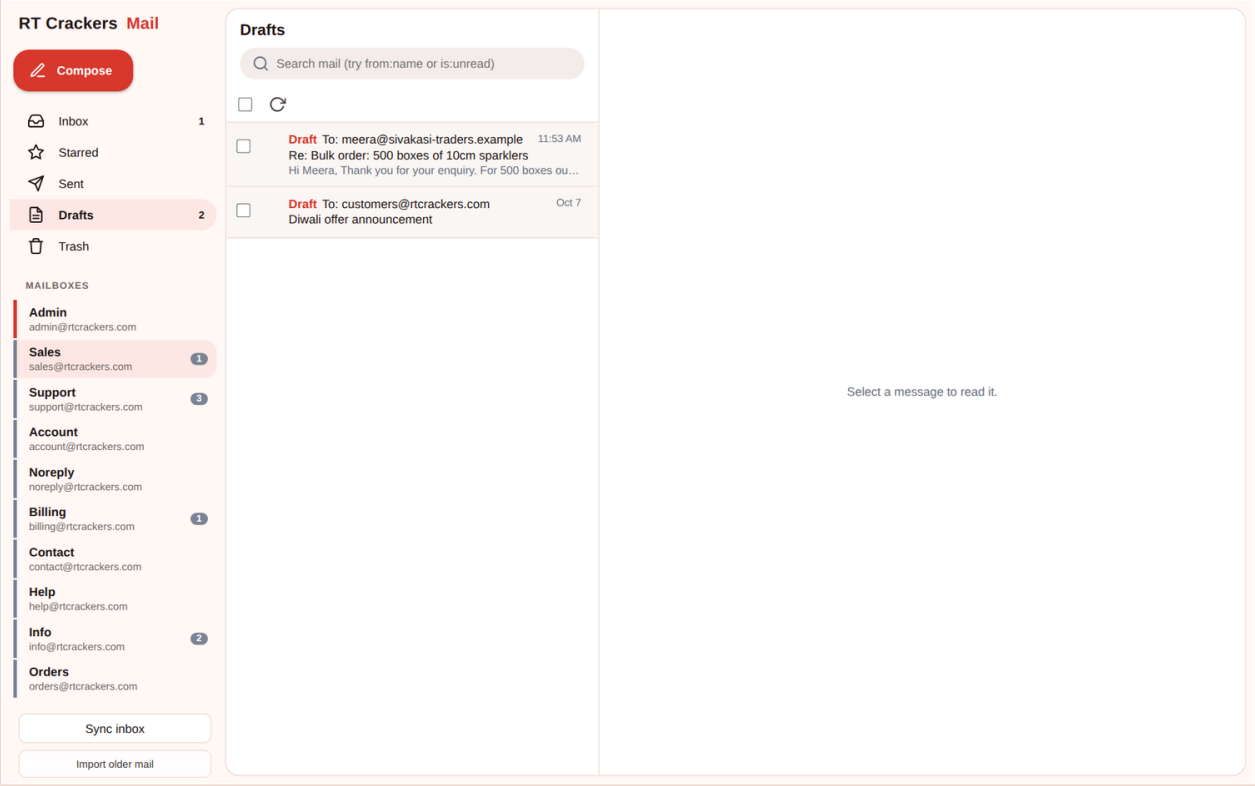


Figure 9. The Drafts folder (red tag, newest edit first).

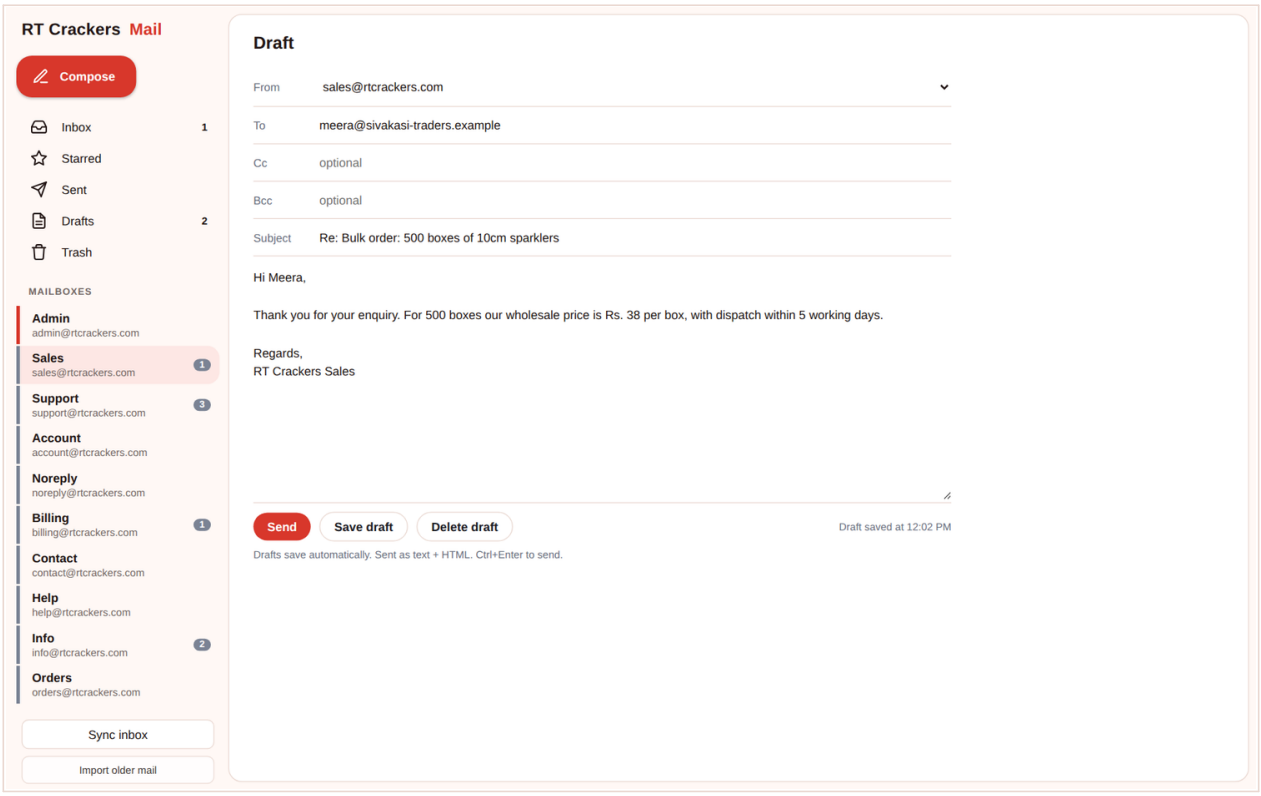


Figure 10. Composing with autosave: the footer shows when the draft was last saved.

Use it	Just start writing - drafts save themselves. Open Drafts to continue later. Save draft saves immediately; Delete draft discards.
Like	Gmail and Outlook Drafts.
Stored in	email_messages with status 'draft' and folder 'drafts'; recipients in email_recipients. No new columns.
Safety	Edits are saved when you navigate away, sign out, or hide the tab; closing the tab with unsaved typing triggers a browser warning; a failed send keeps the draft.

How it works

Each compose session owns a small controller. Saves are **chained**, so two autosaves can never both create a draft, and Send waits for any save in flight so it knows the draft id and removes it afterwards.

public/app.js (lines 639-667) - the compose controller: chained saves

```
const c = { draftId: p.draftId || null, dirty: false, closed: false, timer: null, chain:
    Promise.resolve() };
S.compose = c;
const hasContent = () => [to, cc, bcc, subject, body].some((el) => el.value.trim());
const payload = () => ({
  mailbox: from.value, to: to.value, cc: cc.value, bcc: bcc.value,
  subject: subject.value, body: body.value, reply_to_id: p.replyTo || null, draft_id: c.draftId,
});

c.save = () => {
  c.chain = c.chain.then(async () => {
    if (c.closed || !c.dirty) return;
    c.dirty = false;
    status.textContent = "Saving…";
    try {
      const r = await api("/drafts", { method: "POST", body: JSON.stringify(payload()) });
      if (r.id) {
        if (!c.draftId) bumpDrafts(from.value, 1);
        c.draftId = r.id;
        discard.textContent = "Delete draft";
        status.textContent = "Draft saved at " + new Date().toLocaleTimeString([], { timeStyle:
            "short" });
      } else status.textContent = "";
    } catch (e) {
      c.dirty = true;
      if (/draft not found/i.test(e.message)) c.draftId = null;   // deleted elsewhere: next
          save recreates it
      status.textContent = "Couldn't save draft";
    }
  });
  return c.chain;
};
```

mailcore/service.py (lines 551-587) - server side: create or update a draft

```
    snippet = make_snippet(body, "")
    with conn() as c:
        if draft_id:
            row = c.execute(
                """update email_messages set mailbox_id = %s, from_address = %s, from_name = %s,
                    subject = %s,
                           snippet = %s, text_body = %s, updated_at = now()
                   where email_message_id = %s and folder = 'drafts' and status = 'draft' and
                       deleted_at is null
                     and mailbox_id = any(%s)
                   returning email_message_id, updated_at""",
                (box["mailbox_id"], box["address"], box["label"], subject, snippet, body,
                 int(draft_id), _accessible_ids(user)),
            ).fetchone()
            if not row:
                raise LookupError("Draft not found")
            c.execute("delete from email_recipients where email_message_id = %s",
                (row["email_message_id"],))
        else:
            parent_msg_id = None
            if reply_to_id:
                parent = get_message(user, int(reply_to_id))
                parent_msg_id = (parent or {}).get("message_id")
            row = c.execute(
                """insert into email_messages
                   (mailbox_id, direction, status, in_reply_to, references_header, from_address,
                       from_name, subject,
                    snippet, text_body, is_read, folder)
                   values (%s,'outbound','draft',%s,%s,%s,%s,%s,%s,%s,true,'drafts')
                   returning email_message_id, updated_at""",
                (box["mailbox_id"], parent_msg_id, parent_msg_id, box["address"], box["label"],
                    subject, snippet, body),
            ).fetchone()
        mid = row["email_message_id"]
        for kind, addrs in (("to", to_l), ("cc", cc_l), ("bcc", bcc_l)):
            for a in addrs:
                c.execute(
                    "insert into email_recipients (email_message_id, recipient_type,
                        email_address)"
                    " values (%s, %s::mail_recipient_type, %s)",
                    (mid, kind, a.lower()),
                )
    return {"id": mid, "saved_at": row["updated_at"]}
```

4.6 Reply, Reply all and Forward

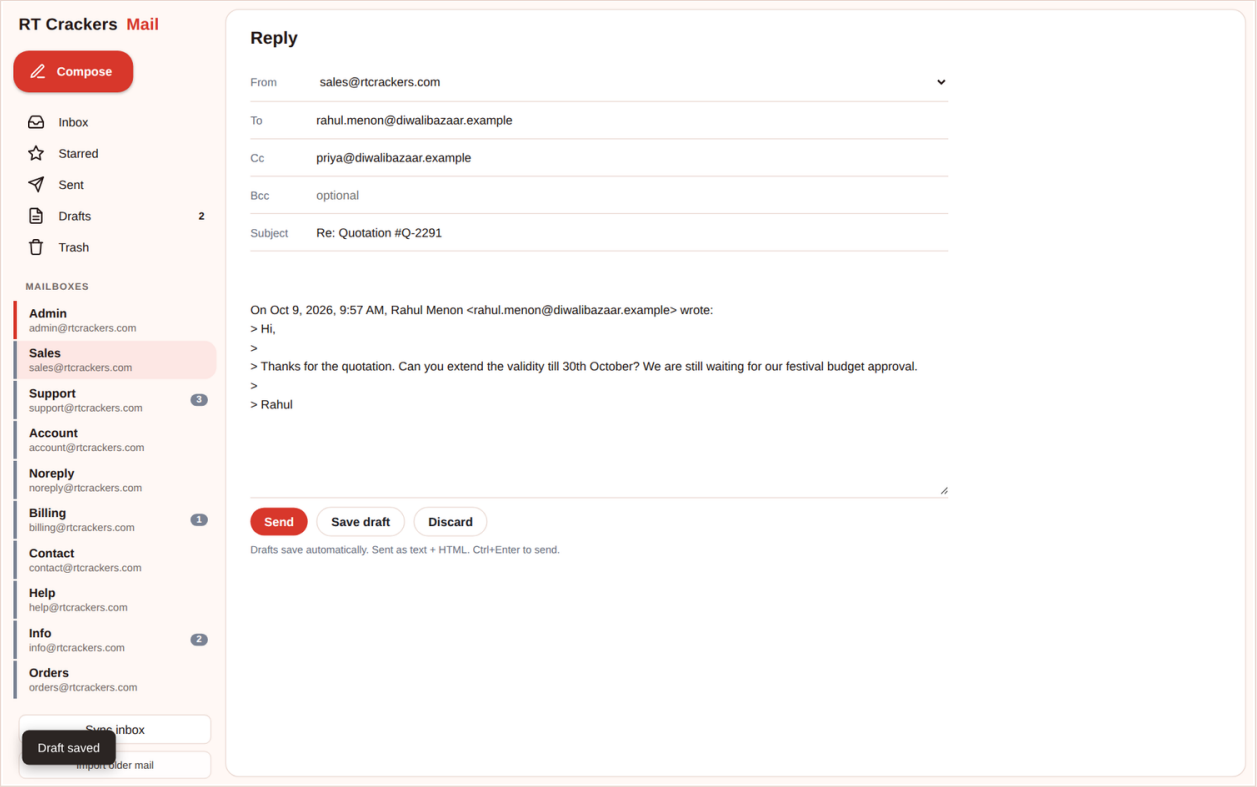


Figure 11. Reply all: sender and other recipients in To, Cc kept, original text quoted, thread headers preserved.

Use it	Open a message and choose Reply, Reply all or Forward (shortcuts R, A, F). Reply all only appears when someone besides you and the sender is on the message.
Like	Gmail and Outlook reply buttons.
Details	Your own address and duplicates are removed; HTML-only mail is converted to clean plain text; a forward starts a new thread without attachments.
Tests	browser: reply-all To/Cc, forward header, HTML-only forward; service: threading headers kept for reply drafts

public/app.js (lines 560-564) - who is still on the message

```
function otherRecipients(m, list) {
  const own = (boxInfo(m.mailbox) || {}).address;
  const skip = new Set([own, m.from_address].filter(Boolean).map((a) => a.toLowerCase()));
  return (list || []).filter((a) => !skip.has(a.toLowerCase()));
}
```

public/app.js (lines 567-577) - Reply and Reply all

```
function reply(m, all = false) {
  const subj = /^re:/i.test(m.subject) ? m.subject : "Re: " + m.subject;
  const p = { from: m.mailbox, to: m.from_address, subject: subj, replyTo: m.id, body:
      quoteOriginal(m) };
  if (all) {
    const to = [...new Set([m.from_address, ...otherRecipients(m, m.to_addrs)])];
    p.to = to.join(", ");
    p.cc = [...new Set(otherRecipients(m, m.cc_addrs).filter((a) => !to.includes(a)))].join(", ");
  }
  enterCompose();
  renderCompose(p);
}
```

4.7 Search operators and Load more

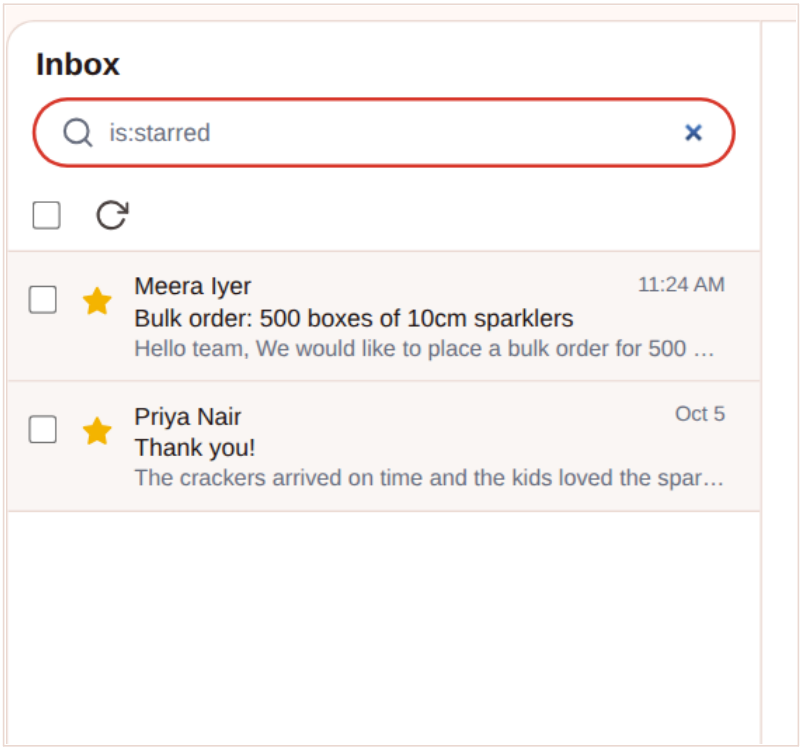


Figure 12. Searching with an operator: is:starred lists only starred messages.

Operator	Meaning	Example
from:	Sender name or address contains	from:meera
to:	A To recipient contains	to:sales@
subject:	Subject contains (quotes allowed)	subject:"price list"
is:unread / is:read	Read state	is:unread
is:starred	Starred messages	is:starred from:anita
free text	Subject, sender, recipients and body	sparklers diwali

How it works

The query is split into operators and free text; every value is passed to SQL as a parameter and wildcard characters are escaped, so a search for % finds a literal percent sign. Results come back 50 at a time; the page requests the next slice with an offset and appends it, skipping any message it already shows.

mailcore/service.py (lines 298-301) - splitting a Gmail-style query

```
def parse_search(q: str) -> tuple[str, list[tuple[str, str]]]:
    """Split a Gmail-style query into free text and operators: from:, to:, subject:,
       is:unread|read|starred."""
    ops = [(m.group(1).lower(), m.group(2).strip('"')) for m in _SEARCH_OP_RE.finditer(q or "")]
    return _SEARCH_OP_RE.sub(" ", q or "").strip(), ops
```

mailcore/service.py (lines 332-357) - applying operators, free text and paging

```
    text, ops = parse_search(q)
    for op, value in ops:
        if op == "from":
            sql += " and (m.from_address ilike %s or m.from_name ilike %s)"
            args += [_like(value)] * 2
        elif op == "to":
            sql += " and exists (select 1 from email_recipients r where r.email_message_id =
                m.email_message_id and r.recipient_type = 'to' and r.email_address ilike %s)"
            args.append(_like(value))
        elif op == "subject":
            sql += " and m.subject ilike %s"
            args.append(_like(value))
        elif op == "is":
            flag = value.lower()
            if flag == "unread":
                sql += " and not m.is_read"
            elif flag == "read":
                sql += " and m.is_read"
            elif flag == "starred":
                sql += " and m.is_starred"
    if text:
        sql += """ and (m.subject ilike %s or m.from_address ilike %s or m.from_name ilike %s or
            m.text_body ilike %s
                   or exists (select 1 from email_recipients r where r.email_message_id =
                       m.email_message_id
                              and r.email_address ilike %s))"""
        args += [_like(text)] * 5
    sql += f" order by {ts} desc, m.email_message_id desc limit %s offset %s"
    args += [limit, offset]
```

public/app.js (lines 279-302) - paging with protection against out-of-order responses

```
async function loadList(append = false) {
  const req = ++S.listReq;   // ignore responses that arrive after a newer request was started
  const offset = append ? S.items.length : 0;
  let items;
  try {
    items = await
        api(`/messages?mailbox=${enc(S.box)}&folder=${S.view}&q=${enc(S.q)}&limit=${PAGE_SIZE}&offset=${offset}`);
  } catch (e) {
    if (req === S.listReq) { S.loading = false; renderRows(); toast(e.message, true); }
    return;
  }
  if (req !== S.listReq || S.view === "compose") return;
  S.loading = false;
  if (append) {
    const have = new Set(S.items.map((m) => m.id));
    S.items = S.items.concat(items.filter((m) => !have.has(m.id)));
  } else {
    S.items = items;
    const ids = new Set(items.map((m) => m.id));
    S.sel = new Set([...S.sel].filter((id) => ids.has(id)));
    S.cur = Math.min(S.cur, items.length - 1);
  }
  S.hasMore = items.length === PAGE_SIZE;
  renderRows();
}
```

4.8 Delivery status

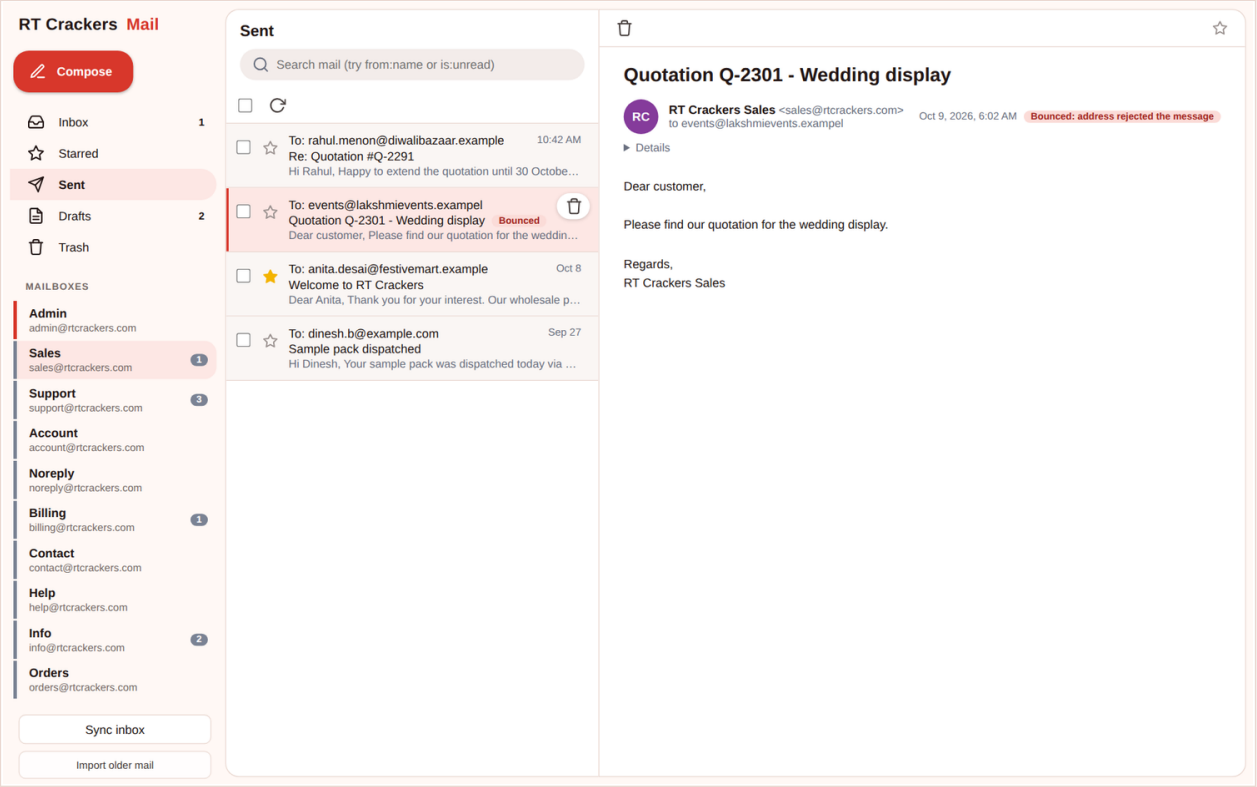


Figure 13. A bounced message is flagged in the list and in the reader header.

Use it	Nothing to do - when Resend reports a result, a chip appears: Delivered, Bounced, Failed or Spam report.
Like	Gmail's "address not found" bounce notice, but shown inline.
Stored in	email_messages.status (existing enum) and resend_event_id.
Setup	In Resend, send delivery events to the same webhook URL as incoming mail: /api/webhooks/resend (same signing secret).

A bad outcome always wins: if a bounce arrives, a later (or replayed) 'delivered' event is ignored, so a real failure can never be hidden.

mailcore/service.py (lines 667-681) - recording delivery results

```
def apply_delivery_event(event_type: str, resend_email_id: str | None, event_id: str | None =
    None) -> int:
    """Record delivered/bounced/... on the matching sent message. Bad outcomes always win; a late
    'delivered' never overwrites a bounce. Returns the number of messages updated (0 if
        unknown)."""
    status = _DELIVERY_EVENTS.get(event_type)
    if not status or not resend_email_id:
        return 0
    with conn() as c:
        rows = c.execute(
            """update email_messages set status = %s::mail_message_status, resend_event_id = %s,
                updated_at = now()
               where resend_email_id = %s and direction = 'outbound'
                 and (status::text in ('sent','queued','delivered') or %s in
                     ('bounced','complained','failed'))
               returning email_message_id""",
            (status, event_id, resend_email_id, status),
        ).fetchall()
    return len(rows)
```

mailcore/routes.py (lines 330-334) - the webhook branch (signature already verified above it)

```
    if event.get("type") in service._DELIVERY_EVENTS:
        updated = await run_in_threadpool(
            service.apply_delivery_event, event["type"], data.get("email_id"),
                headers.get("svix-id")
        )
        return {"delivery": event["type"], "updated": updated}
```

4.9 Privacy: remote images blocked

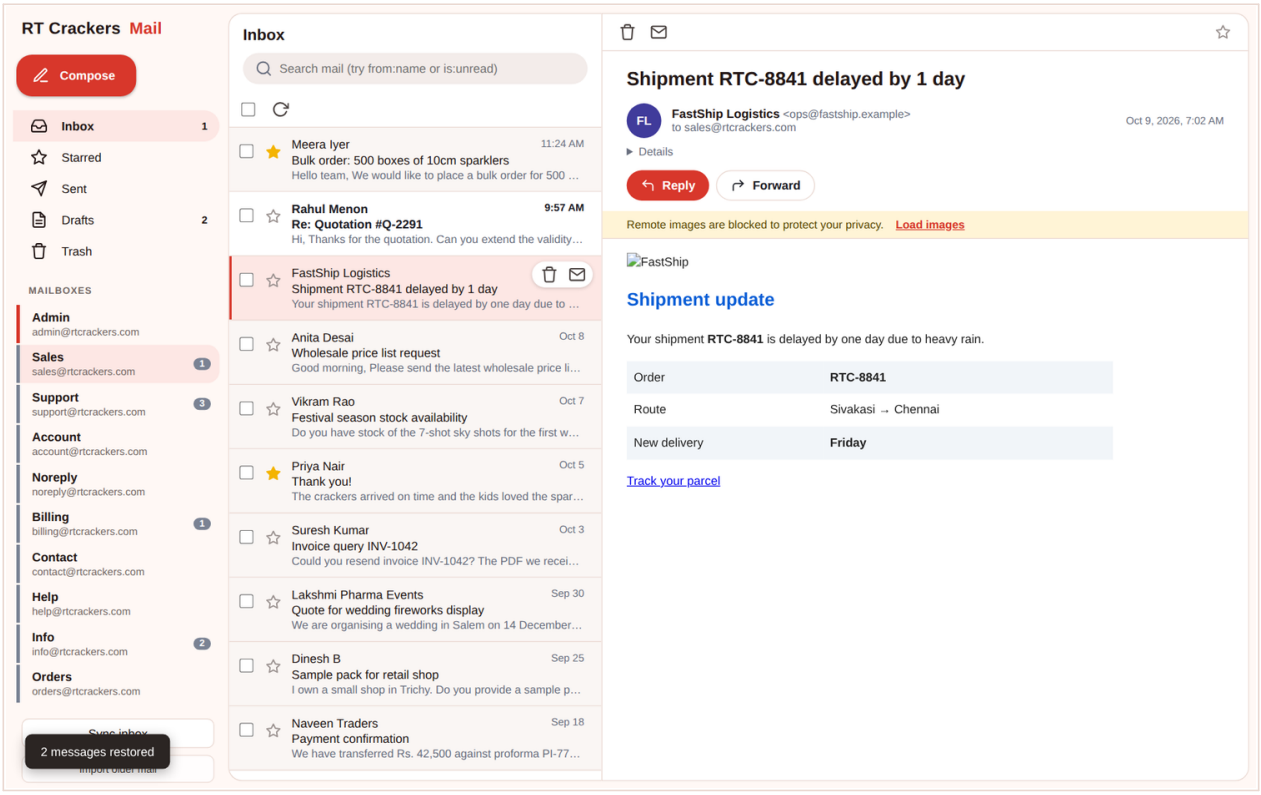


Figure 14. An HTML email with a banner image and a hidden tracking pixel. Nothing is fetched until you choose Load images.

Why	Marketing emails embed tiny remote images that tell the sender when and where you opened a message.
What happens	HTML mail is rendered in a sandboxed iframe with scripts disabled. Its Content-Security-Policy allows only embedded (data:) images until you click Load images for that one message.
Verified	In a real Chromium run the browser refused both the banner and the tracking pixel (Content-Security-Policy console messages).

public/app.js (lines 516-532) - images blocked by default; opt in per message

```
    const hasRemote = REMOTE_IMG_RE.test(m.html_body);
    const frameBody = h("div", { class: "msg-body framed" });
    const renderFrame = (allowImages) => {
      const imgSrc = allowImages ? "https: data:" : "data:";
      const doc = `<!doctype html><meta charset="utf-8"><base target="_blank"><meta
          name="referrer" content="no-referrer">` +
        `<meta http-equiv="Content-Security-Policy" content="default-src 'none'; img-src
            ${imgSrc}; style-src 'unsafe-inline'">` +
        `<style>body{font:14px/1.55 system-ui,sans-serif;margin:16px
            28px;color:#222}img{max-width:100%;height:auto}</style>` + m.html_body;
      frameBody.replaceChildren(
        h("iframe", { class: "mail-frame", sandbox: "allow-popups allow-popups-to-escape-sandbox",
             srcdoc: doc, title: "Message" }));
    };
    renderFrame(false);   // remote images (tracking pixels) are blocked until the user opts in
    if (hasRemote) {
      const imgBar = h("div", { class: "img-bar", role: "status" },
        h("span", {}, "Remote images are blocked to protect your privacy."),
        h("button", { type: "button", class: "link", onclick: () => { renderFrame(true);
            imgBar.remove(); } }, "Load images"));
      body = h("div", { class: "msg-wrap" }, imgBar, frameBody);
    } else body = frameBody;
```

main.py (lines 29-35) - response headers on every request

```
SECURITY_HEADERS = {
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
    "Referrer-Policy": "same-origin",
    "Permissions-Policy": "camera=(), microphone=(), geolocation=()",
    "Content-Security-Policy": "script-src 'self'; object-src 'none'; frame-ancestors 'none';
        form-action 'self'",
}
```

4.10 Keyboard shortcuts

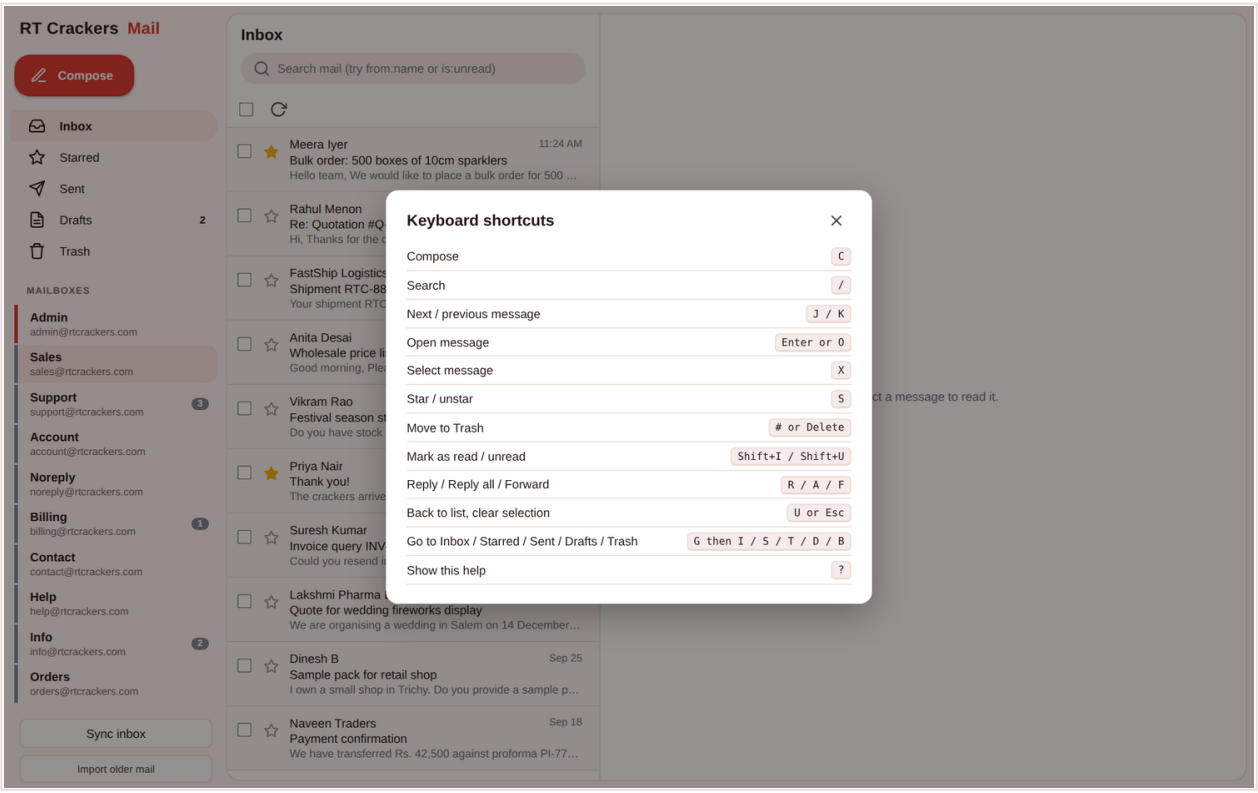


Figure 15. Press ? for the built-in help.

Key	Action
C	Compose
/	Focus search
J / K	Next / previous message
Enter or O	Open message
X	Select message
S	Star / unstar
# or Delete	Move to Trash (Delete forever inside Trash)
Shift+I / Shift+U	Mark as read / unread
R / A / F	Reply / Reply all / Forward
U or Esc	Back to the list / clear selection
G then I, S, T, D, B	Go to Inbox, Starred, Sent, Drafts, Trash
?	Show shortcut help

Shortcuts are ignored while you type in a field or while the compose form is open, so they can never trigger by accident.

public/app.js (lines 952-991) - the shortcut handler (first part)

```
function onKey(e) {
  if (e.ctrlKey || e.metaKey || e.altKey) return;
  const t = e.target, tag = t && t.tagName;
  if (tag === "INPUT" || tag === "TEXTAREA" || tag === "SELECT" || (t && t.isContentEditable)) {
    if (e.key === "Escape" && t.id === "search") t.blur();
    return;
  }
  if (!S.cfg || !$("#login").hidden) return;
  if (!$("#help").hidden) { if (e.key === "Escape" || e.key === "?") toggleHelp(false); return; }
  const k = e.key;
  if (gPending) {
    gPending = false;
    clearTimeout(gTimer);
    const v = { i: "inbox", s: "starred", t: "sent", d: "drafts", b: "trash" }[k.toLowerCase()];
    if (v) { e.preventDefault(); setView(v); }
    return;
  }
  if (S.view === "compose") return;   // the compose form has its own controls
  const open = S.openMsg;
  switch (k) {
    case "?": toggleHelp(true); break;
    case "g": gPending = true; gTimer = setTimeout(() => (gPending = false), 1200); break;
    case "c": e.preventDefault(); setView("compose"); break;
    case "/": e.preventDefault(); $("#search").focus(); break;
    case "j": moveCur(1); break;
    case "k": moveCur(-1); break;
    case "Enter": if (t === document.body && S.items[S.cur]) { const m = S.items[S.cur]; (S.view
        === "drafts" ? openDraft : openMessage)(m.id); } break;
    case "o": if (S.items[S.cur]) { const m = S.items[S.cur]; (S.view === "drafts" ? openDraft :
        openMessage)(m.id); } break;
    case "x": if (S.items[S.cur]) toggleSel(S.items[S.cur].id, !S.sel.has(S.items[S.cur].id));
        break;
    case "s": { const id = S.openId || (S.items[S.cur] && S.items[S.cur].id); if (id && S.view !==
         "drafts") toggleStar(id); break; }
    case "#": case "Delete": bulk(S.view === "trash" ? "delete_forever" : S.view === "drafts" ?
        "delete_draft" : "trash", targetIds()); break;
    case "I": if (S.view !== "trash" && S.view !== "drafts") bulk("read", targetIds()); break;
    case "U": if (S.view !== "trash" && S.view !== "drafts") bulk("unread", targetIds()); break;
    case "u": if (S.openId) closeReader(); break;
    case "r": if (open && open.folder === "inbox" && !open.deleted) reply(open); break;
    case "a": if (open && open.folder === "inbox" && !open.deleted && hasOthers(open)) reply(open,
         true); break;
    case "f": if (open && open.folder === "inbox" && !open.deleted) forward(open); break;
    case "Escape": if (S.sel.size) selectAll(false); else if (S.openId) closeReader(); break;
  }
}
```

4.11 Phone layout

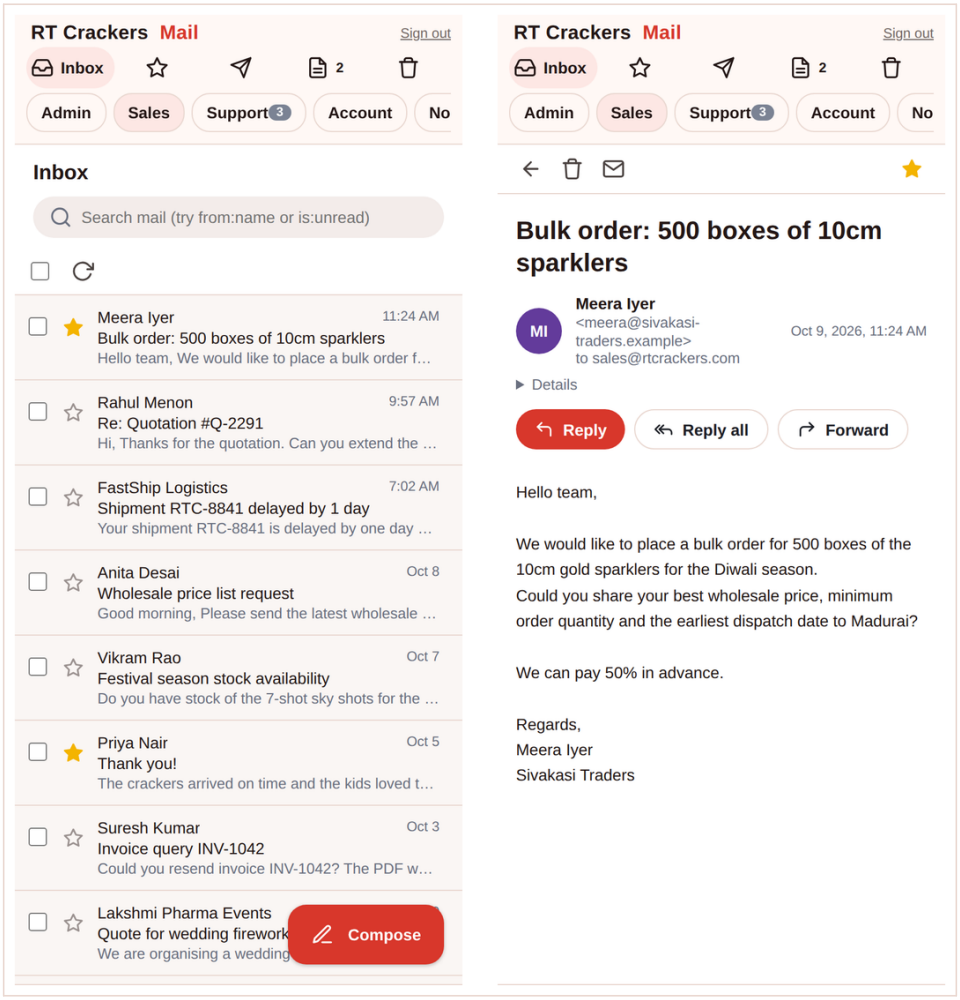


Figure 16. On a phone: folder tabs and mailbox chips on top, one pane at a time, Back button in the reader, floating Compose.

Below 900 px the three panes collapse into one. Opening a message adds a `reading` class that swaps the list for the reader; the Back arrow removes it. Hover actions are hidden (they need a mouse); checkboxes, stars and the bulk toolbar work with touch.

public/app.js (lines 450-456) - Back button

```
function closeReader() {
  S.openId = null;
  S.openMsg = null;
  $("#app").classList.remove("reading");
  markRowOpen(null);
  readerPlaceholder();
}
```

4.12 Mailboxes, login and access control

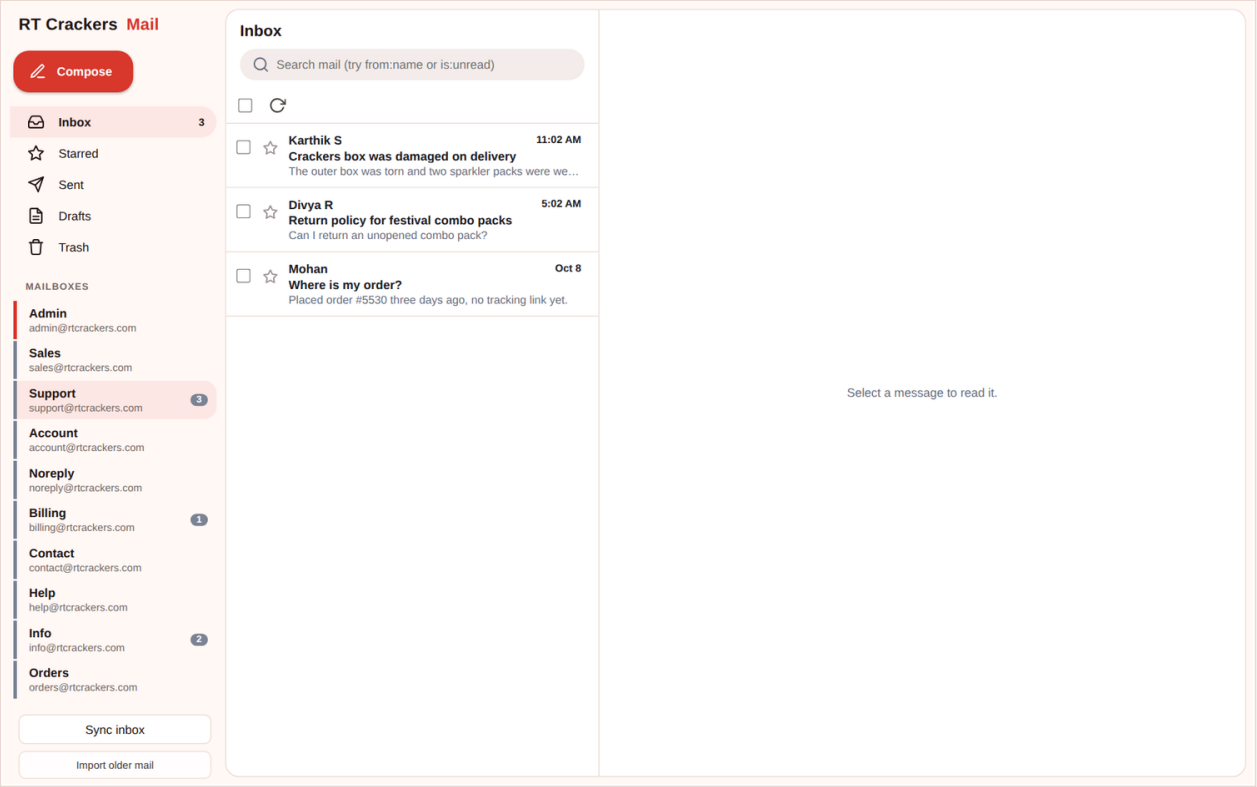


Figure 17. Switching to the support mailbox keeps you in the same folder; badges show unread mail per mailbox.

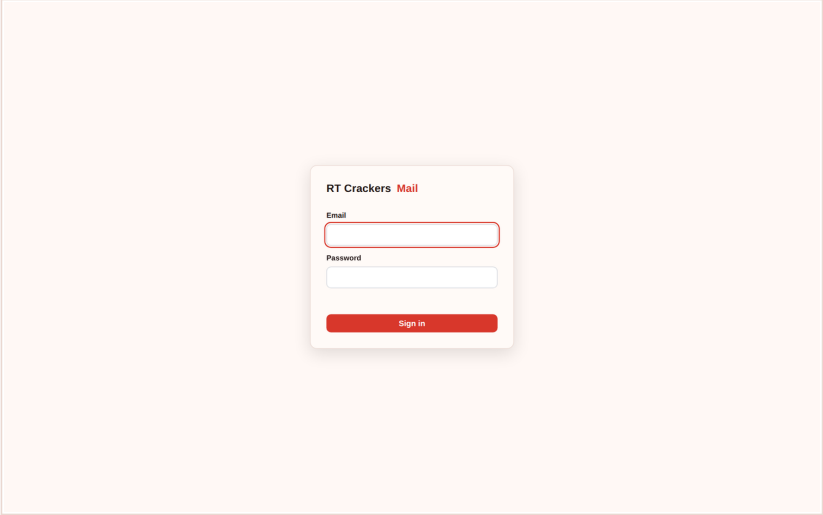


Figure 18. Sign-in screen (Supabase Auth). The portal admin account is configurable with MAIL_ADMIN_EMAIL.

- Every database query is scoped to the mailboxes the signed-in user may access, so a user cannot read, star, trash or edit another mailbox's mail even with a guessed id (covered by tests).
- Access tokens are cached for 60 seconds in a thread-safe, size-limited cache; concurrent refreshes share one request so users are not signed out by token rotation.
- Private API responses are sent with Cache-Control: no-store.

4.13 Speed: why the site loads fast now

Each database call used to open a brand-new connection to Supabase, and almost every screen needed several of them. On a host where the database is far away, every connection costs a network round trip, so the cost was the **number of connections**, not the amount of data. The fixes keep every SQL statement and the database layer unchanged and simply need far fewer connections.

Action	Connections before	Connections after
Open the site (6 requests became 2)	10	2
Background sync of 100 already-stored emails	102	1
Open one message (including count refreshes)	6	1
Trash a message (including count refreshes)	6	1

Measured in real Chromium against a real PostgreSQL with 150 ms of delay added to every connection (to imitate a remote database): the first list appeared after **1,402 ms** before and **297 ms** after. These are controlled measurements on a test machine; real results also depend on the Vercel function region, the Supabase region and cold starts.

mailcore/service.py (lines 708-717) - everything the first screen needs, in one connection

```
def bootstrap(user, local: str | None = None, folder: str = "inbox", limit: int = 50) -> dict:
    """Everything the first screen needs, in one connection: mailboxes, counts, and the first page
        of a folder."""
    ensure_mailboxes()
    boxes = list_mailboxes(user)
    chosen = next((b for b in boxes if b["local_part"] == local), None) or (boxes[0] if boxes else
        None)
    ids = [b["mailbox_id"] for b in boxes]
    with conn() as c:
        data = _pulse(c, ids)
        items = _list_messages(c, chosen, chosen["local_part"], folder, "", limit, 0) if chosen
            else []
    return {"boxes": boxes, "mailbox": chosen["local_part"] if chosen else "", "items": items,
        **data}
```

mailcore/service.py (lines 205-226) - one query tells a sync which emails are already stored

```
def _load_known(rids: list[str]) -> dict:
    """Which of these Resend emails are already stored, and for which mailboxes? One query for any
        number of ids.

    Returns {resend_id: (set_of_mailbox_ids, base_id_is_used)}. A message delivered to two of our
        mailboxes is
    stored as <id> and <id>#<mailbox_id>, so both forms are matched on the part before the '#'.
    """
    rids = [r for r in rids if r]
    if not rids:
        return {}
    with conn() as c:
        rows = c.execute(
            "select mailbox_id, resend_email_id from email_messages where
                split_part(resend_email_id, '#', 1) = any(%s)",
            (rids,),
        ).fetchall()
    out: dict = {}
    for r in rows:
        stored_id = r["resend_email_id"]
        base = stored_id.split("#", 1)[0]
        mailboxes, used = out.get(base, (set(), False))
        mailboxes.add(r["mailbox_id"])
        out[base] = (mailboxes, used or stored_id == base)
    return out
```

public/app.js (lines 816-824) - new mail shows up through a tiny check

```
async function pollPulse() {
  if (document.hidden || !S.cfg) return;
  let p;
  try { p = await api("/pulse"); } catch { return; }
  if (applyPulse(p)) {
    toast("New mail");
    if (S.view === "inbox" && !S.q) loadList();
  }
}
```

5. API reference

Method	Path	Purpose	Sign-in
GET	`/api/health`	Liveness probe (no database access).	No
POST	`/api/auth/login`	Sign in with email and password.	No
POST	`/api/auth/refresh`	Exchange a refresh token for a new access token.	No
GET	`/api/theme`	Public branding for the login screen (cached 5 min).	No
GET	`/api/company`	Public company name and logo (cached 5 min).	No
GET	`/api/branding`	Theme and company logo in one request (cached; public).	No
GET	`/api/bootstrap`	First screen in one request: mailboxes, counts, drafts, newest ids, first page of a folder.	Yes
GET	`/api/pulse`	Cheap new-mail check: unread counts, draft counts, newest inbox id per mailbox.	Yes
GET	`/api/config`	Mailboxes the user can access, user email, Resend status.	Yes
GET	`/api/counts`	Unread count per mailbox.	Yes
GET	`/api/counts/drafts`	Draft count per mailbox.	Yes
GET	`/api/messages`	List messages: mailbox, folder (inbox, sent, drafts, starred, trash), q, limit (max 200), offset.	Yes
GET	`/api/messages/{id}`	One message; marks inbox mail read (not when in Trash).	Yes
POST	`/api/messages/bulk`	Apply read, unread, star, unstar, trash, restore or delete_forever to up to 200 ids.	Yes
POST	`/api/trash/empty`	Permanently delete everything in one mailbox's Trash.	Yes
POST	`/api/send`	Send a message (optionally from a draft or as a reply).	Yes
POST	`/api/drafts`	Create a draft, or update the one named by draft_id.	Yes
DELETE	`/api/drafts/{id}`	Delete a draft.	Yes
POST	`/api/sync`	Pull received mail from Resend (add ?full=1 to import older mail).	Yes
POST	`/api/webhooks/resend`	Resend webhook: email.received plus delivered/bounced/complained/failed. Verified by Svix signature.	Signature

6. Database usage - no schema changes

The brief was to enhance the code base without altering the database integrations. Every new feature maps onto a column or enum value that the original schema already provides. `mailcore/db.py`, `supabase/*.sql` and `tests/schema.sql` are byte-identical to the uploaded originals.

Feature	Table.column / value used
Read / unread	email_messages.is_read
Star	email_messages.is_starred
Trash, restore, delete forever	email_messages.deleted_at (soft delete); permanent delete removes the row and cascades to email_recipients
Drafts	email_messages.status = 'draft', folder = 'drafts'; recipients in email_recipients (to / cc / bcc)
Delivery status	email_messages.status in ('delivered','bounced','complained','failed'), resend_event_id
Reply threading	email_messages.in_reply_to, references_header, message_id
Mailboxes and permissions	mailboxes, mailbox_access, admin_access, business_email_addresses (unchanged queries)

Search and listing queries read the same tables with additional filters; all values are bound parameters.

7. Quality and testing

Layer	Count	What it proves
Unit tests (test_unit.py)	32	Date parsing, snippets, address parsing, Resend retry, config, auth helpers - no database or network.
Service tests (test_service_db.py)	52	Real PostgreSQL built from tests/schema.sql: sync, send, drafts, stars, trash, bulk permissions, search operators, paging, delivery status.
API tests (test_api.py)	57	Routing, validation, authentication, security headers, webhook signatures, draft and bulk endpoints.
Browser-logic checks (smoke.js)	54	The real app.js in jsdom against a fake API: autosave, resume, send, undo, paging, shortcuts, mobile back flow.

Real-browser verification

The finished app was also run in headless Chromium against the real FastAPI server and a real PostgreSQL database loaded with sample mail; every screenshot in this guide comes from that run. This found two defects that the simulated tests could not see, both fixed and now guarded by regression checks:

- A stray word **"null"** was printed in the reader for messages outside Trash (a missing banner element was passed to the DOM).
- A harsh red **keyboard-cursor outline** appeared after an ordinary mouse click; it now shows only while the keyboard is in use.
- The phone header was cramped and had no sign-out; it is now a header row, an evenly spaced folder row and scrollable mailbox chips.

Running the tests

```
python -m pytest -q                                  # unit + API tests; DB tests skip without Postgres
TEST_DATABASE_URL=postgresql://user:pass@localhost/rtmail_test python -m pytest -q
cd tests/frontend && npm install && npm test         # browser-logic checks (jsdom)
```

Never point TEST_DATABASE_URL at your real Supabase database - the test schema drops and recreates tables.

8. Setup and deployment

Variable	Purpose
DATABASE_URL	Postgres connection (Supabase). Server-side only.
SUPABASE_URL, SUPABASE_ANON_KEY	Supabase Auth.
RESEND_API_KEY	Sending and receiving mail.
RESEND_WEBHOOK_SECRET	Verifies webhook signatures (received + delivery events).
MAIL_DOMAIN	Company mail domain (default rtcrackers.com).
MAIL_ADMIN_EMAIL	The one account allowed to sign in (default admin@MAIL_DOMAIN).
SYNC_BUDGET_SECONDS	Time limit for one sync (default 45).
MAIL_DEV_NO_AUTH=1	Local development only; ignored on Vercel.

Deploy

- Push the folder to GitHub and import it in Vercel (no build command). Add the variables above.
- Sign in as the Supabase user admin@your-domain, then use **Import older mail** once.
- In Resend, point the webhook at `https://your-domain/api/webhooks/resend` and enable **email.received** plus the delivery events (delivered, bounced, complained, failed).
- Local run: `pip install -r requirements-dev.txt`, copy .env.example to .env, then `uvicorn main:app --reload`.

9. Limitations and suggested next steps

Next step	Why it matters	Database impact
Attachments (send and view)	Biggest remaining gap versus Gmail and Outlook.	Sending needs none; storing copies needs Storage or a new table.
Conversation view	Group replies into one thread.	None for display (headers exist); better matching may need a thread key.
Labels and archive	Organise beyond folders.	New table or columns.
Faster search at scale	ilike over bodies slows down on very large mailboxes.	A small additive index migration.
Rich-text composer and signatures	Formatted replies.	Optional (signature per user).
Dark mode, undo send, drag and drop	Comfort and polish.	None.

Appendix: project files

Path	Lines
`main.py`	101
`mailcore/routes.py`	345
`mailcore/service.py`	718
`mailcore/auth.py`	144
`mailcore/resend_client.py`	92
`mailcore/db.py`	46
`mailcore/config.py`	42

Path	Lines
public/index.html	68
public/app.js	1025
public/style.css	214
tests/test_unit.py	170
tests/test_service_db.py	567
tests/test_api.py	387
tests/frontend/smoke.js	312

Other files: README.md, ENVIRONMENT.md, .env.example, requirements.txt, requirements-dev.txt, pyproject.toml, supabase/*.sql (unchanged), tests/schema.sql (unchanged), docs/ (this guide and the screenshots).